Lectura ejecutiva del documento 12

Tema	Definición	Decisión de alto impacto
Objetivo	Fijar el contrato exacto de acciones y endpoints del MVP	Que frontend, backend, QA y producto compartan una sola verdad
Stack contextual	Next.js App Router + API Routes/Server Actions + Supabase	Ergonomía de desarrollo sin perder contrato estable
Ámbito	Dashboard, tasks, boards, scores, reports, checklists, alerts y settings	Nada de endpoints fantasmas ni payloads mutantes
Regla madre	Mismo DTO para Server Action y endpoint HTTP	Menos drift, menos bugs, menos sorpresas
Salida final	Payloads, respuestas, errores, validaciones, auth y códigos de estado	Base directa para construir, testear y versionar

1. Objetivo del documento

Definir el contrato técnico exacto de acciones y endpoints del MVP operativo de Capataz para que frontend, backend, QA y producto trabajen sobre la misma superficie.

Capataz necesita un contrato técnico con dientes. El frontend puede correr rápido, pero no puede inventarse semántica por su cuenta ni hacer que cada endpoint sea una criatura de laboratorio distinta.

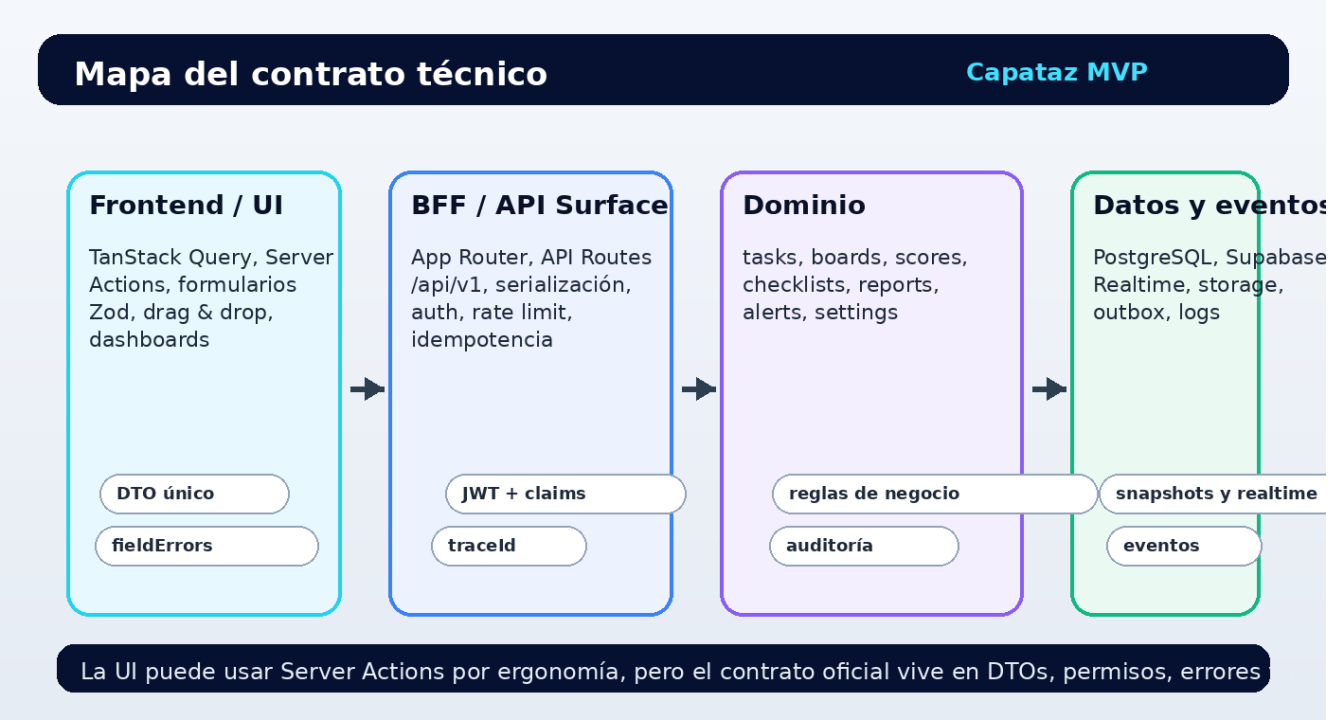
2. Filosofía del contrato

Principios del contrato

- Una acción de servidor y su endpoint HTTP equivalente deben compartir el mismo DTO, las mismas validaciones, los mismos permisos y la misma semántica de error.
- La API oficial del MVP vive bajo `/api/v1`.
- Los identificadores del dominio son UUID v4 en formato string.
- Las fechas salen y entran en ISO 8601 UTC.
- Las respuestas exitosas usan el sobre `data + meta`; los errores usan `error + traceId`.
- Las mutaciones críticas aceptan `Idempotency-Key` para evitar duplicados de red o doble click.

Auth y transporte

- Clientes web SSR/Server Actions: sesión de Supabase resuelta en servidor y claims de rol.
- Clientes HTTP o testing: `Authorization: Bearer <jwt>`.
- Cada request autorizado lleva `X-Workspace-Id` cuando el contexto no se deduce de la sesión.
- El rol efectivo se cruza con políticas RLS y validaciones de backend; jamás se confía en el frontend.



3. Formato común de respuestas

Todas las respuestas exitosas viajan con `data` y `meta`. Todos los errores viajan con `error` y `traceId`. Esto le da a UI, logs, QA y soporte una gramática compartida.

Éxito

```
{
  "data": {},
  "meta": {
    "traceId": "tr_01JX..."
  }
}
```

Error

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "The request payload is invalid.",
    "fieldErrors": {
      "title": [
        "Title is required."
      ],
      "dueAt": [
        "Due date must be later than assignedAt."
      ]
    },
    "traceId": "tr_01JX6D8F7J17R8Q66Y2N5S4M2C"
  }
}
```

4. Códigos de estado

Código	Semántica
200 OK	Lectura o actualización exitosa
201 Created	Recurso creado
202 Accepted	Trabajo asíncrono aceptado
204 No Content	Mutación mínima sin payload grande
400 Bad Request	Payload o query mal formado
401 Unauthorized	Falta o expira la sesión
403 Forbidden	La sesión existe, pero no tiene permiso
404 Not Found	El recurso no existe o no pertenece al workspace
409 Conflict	Colisión de estado, orden o idempotencia
422 Unprocessable Entity	Validación de dominio falló
429 Too Many Requests	Rate limiting o protección anti abuso
500 Internal Server Error	Fallo inesperado con `traceId`

5. Catálogo MVP de endpoints

Esta es la superficie mínima para el dashboard operativo estilo Monday, pero con armadura LotOS y decisiones más sobrias. Los paths van versionados desde el día uno.

Método	Path	Módulo	MVP	Auth
GET	/api/v1/dashboard/summary	Dashboard	Sí	admin, gerente, supervisor
GET	/api/v1/dashboard/feed	Dashboard	Sí	admin, gerente, supervisor, operador
GET	/api/v1/tasks	Tasks	Sí	todos
POST	/api/v1/tasks	Tasks	Sí	admin, gerente, supervisor
PATCH	/api/v1/tasks/:id	Tasks	Sí	admin, gerente, supervisor
POST	/api/v1/tasks/:id/status	Tasks	Sí	admin, gerente, supervisor, operador
GET	/api/v1/boards/:id	Boards	Sí	todos
POST	/api/v1/boards/:id/columns	Boards	Sí	admin, gerente
POST	/api/v1/boards/:id/cards/reorder	Boards	Sí	admin, gerente, supervisor
GET	/api/v1/scores/summary	Scores	Sí	admin, gerente, supervisor
POST	/api/v1/scores/recalculate	Scores	Sí	admin, gerente
GET	/api/v1/reports	Reports	Sí	admin, gerente, supervisor
POST	/api/v1/reports/generate	Reports	Sí	admin, gerente, supervisor
GET	/api/v1/checklists/instances	Checklists	Sí	todos
PATCH	/api/v1/checklists/instances/:id/items/:itemId	Checklists	Sí	todos con ownership/permiso
GET	/api/v1/alerts	Alerts	Sí	todos
PATCH	/api/v1/alerts/:id/read	Alerts	Sí	todos
GET	/api/v1/settings/	Settings	Sí	admin, gerente

	workspace			

6. Mapeo de Server Actions a contrato HTTP

La UI puede usar Server Actions por ergonomía y composición con React Server Components. Aun así, el contrato oficial queda definido a nivel DTO + permisos + errores + respuesta. La acción es la máscara; el contrato es el esqueleto.

Server Action	HTTP equivalente	Uso
getDashboardSummaryAction	GET /dashboard/summary	lectura SSR/Home
listTasksAction	GET /tasks	tabla, filtros, paginación
createTaskAction	POST /tasks	alta desde modal o quick add
updateTaskAction	PATCH /tasks/:id	edición inline o modal
changeTaskStatusAction	POST /tasks/:id/status	drag & drop o cambio manual
createBoardColumnAction	POST /boards/:id/columns	configuración de Kanban
reorderBoardCardsAction	POST /boards/:id/cards/reorder	persistencia de orden
getScoreSummaryAction	GET /scores/summary	widgets y ranking
recalculateScoreAction	POST /scores/recalculate	botón admin / job
generateReportAction	POST /reports/generate	exportación
toggleChecklistItemAction	PATCH /checklists/instances/:id/items/:itemId	check operativo
markAlertReadAction	PATCH /alerts/:id/read	centro de alertas

7. Contratos detallados por endpoint

Cada contrato entra con propósito, auth, payloads, respuesta, validaciones y matriz de errores. Esto evita que el backend crezca como feria sin mapa.

GET /api/v1/dashboard/summary · Dashboard summary

Propósito	Entregar KPIs de cabecera, tareas prioritarias, score cards y tendencia semanal de la Home.
Auth requerida	Requiere sesión válida. Roles: admin, gerente, supervisor. Operador queda fuera en MVP.
Query / params	`{ "range": "7d | 14d | 28d", "workspaceId": "uuid opcional si no viene por sesión" }`
Request body	No aplica
Response ejemplo	`{ "data": { "metrics": { "openTasks": 42, "overdueTasks": 7, "completedToday": 19, "teamScore": 81.4 }, "priorityTasks": [{ "id": "tsk_123", "title": "Abrir sucursal Centro", "priority": "URGENT", "dueAt": "2026-03-13T15:00:00Z", "assignee": { "id": "usr_1", "name": "Valeria Soto" } }], "weeklyTrend": [{ "label": "Mon", "completed": 14, "created": 17 }, { "label": "Tue", "completed": 20, "created": 12 }], "scoreCards": [{ "bucket": "high", "count": 11 }, { "bucket": "medium", "count": 5 }, { "bucket": "low", "count": 2`

	`}`
	`]`
	`},`
	`"meta": {`
	`"range": "7d",`
	`"generatedAt": "2026-03-12T22:20:10Z"`
	`}`
	`}`

Status codes	200, 401, 403, 404, 500
Errores esperados	401 si no hay sesión; 403 si el rol no puede ver Home; 404 si el workspace no existe; 500 con `traceId`

Validaciones clave

- `range` debe ser uno de `7d`, `14d`, `28d`.
- El workspace debe pertenecer a la sesión o venir por claim efectivo.

GET /api/v1/tasks · List tasks

Propósito	Listar tareas en tabla o vista filtrada para Home, módulo de Tareas y selector de Kanban.
Auth requerida	Todos los roles autenticados. El resultado se recorta por permisos y ownership.

Query / params	`{`
	`"page": 1,`
	`"pageSize": 20,`
	`"status": "TODO|IN_PROGRESS|BLOCKED|DONE",`
	`"priority": "LOW|MEDIUM|HIGH|URGENT",`
	`"assigneeId": "uuid",`
	`"search": "texto libre",`
	`"sortBy": "dueAt|priority|createdAt",`
	`"sortOrder": "asc|desc"`
	`}`

Request body	No aplica

Response ejemplo	`{`
	`"data": [`
	`{`
	`"id": "tsk_123",`
	`"title": "Inventario apertura",`
	`"status": "IN_PROGRESS",`
	`"priority": "HIGH",`
	`"dueAt": "2026-03-12T23:00:00Z",`
	`"boardColumnId": "col_doing",`
	`"assignee": {`
	`"id": "usr_2",`
	`"name": "Víctor Pérez"`
	`}`
	`}`
	`],`
	`"meta": {`
	`"page": 1,`
	`"pageSize": 20,`
	`"total": 145,`
	`"hasNextPage": true`
	`}`
	`}`

Status codes	200, 400, 401, 403, 500

Errores esperados	400 por query inválida; 401 sin sesión; 403 por acceso cruzado; 500 inesperado

Validaciones clave

- `page` >= 1 y `pageSize` entre 1 y 100.
- `sortBy` y `sortOrder` limitados a valores controlados.
- El operador solo ve tareas propias o permitidas por su rol.

POST /api/v1/tasks · Create task

Propósito	Crear una tarea operativa con asignación inicial, prioridad y fecha objetivo.
Auth requerida	Roles: admin, gerente, supervisor.
Request body	`{ "title": "Revisión de caja turno mañana", "description": "Validar corte, evidencias y firma del supervisor.", "priority": "HIGH", "status": "TODO", "boardId": "brd_ops", "boardColumnId": "col_todo", "assigneeId": "usr_2", "dueAt": "2026-03-13T14:00:00Z", "tags": [ "apertura", "sucursal-centro" ] }`
Response ejemplo	`{ "data": { "id": "tsk_new_001", "title": "Revisión de caja turno mañana", "status": "TODO", "priority": "HIGH", "createdAt": "2026-03-12T22:24:10Z" }, "meta": { "created": true } }`
Status codes	201, 400, 403, 404, 409, 422, 500
Errores esperados	400 payload mal formado; 403 sin permiso; 404 board/assignee no encontrado; 409 idempotency o duplicado funcional; 422 validación de dominio

Validaciones clave

- `title` requerido, 3 a 160 caracteres.
- `priority` y `status` son enums cerrados.
- `dueAt` no puede ser anterior a `now - 5m`.
- `assigneeId` debe pertenecer al workspace.
- Si se envía `boardColumnId`, debe pertenecer al `boardId`.

PATCH /api/v1/tasks/:id · Update task

Propósito	Editar una tarea existente sin cambiar su estado operativo principal.
Auth requerida	Roles: admin, gerente, supervisor. Operador solo si se habilita edición parcial futura.
Request body	`{ "title": "Revisión de caja turno mañana - corregida", "priority": "URGENT", "dueAt": "2026-03-13T13:00:00Z", "description": "Nueva observación validada por gerencia." }`
Response ejemplo	`{ "data": { "id": "tsk_123", "updatedFields": [ "title", "priority", "dueAt", "description" ], "updatedAt": "2026-03-12T22:28:54Z" }, "meta": { "version": 7 } }`
Status codes	200, 401, 403, 404, 409, 422, 500
Errores esperados	401; 403; 404; 409 por versión/optimistic lock; 422

Validaciones clave

- Solo se aceptan campos whitelisteados.
- No se puede editar una tarea archivada o eliminada lógicamente.
- `dueAt` debe seguir siendo coherente con la política del workspace.

POST /api/v1/tasks/:id/status · Change task status

Propósito	Mover una tarea entre estados o columnas, ya sea por drag & drop o cambio manual.
Auth requerida	Roles: admin, gerente, supervisor, operador con permiso sobre la tarea.
Request body	`{ "status": "DONE", "boardColumnId": "col_done", "reason": "completed", "movedBy": "usr_2" }`
Response ejemplo	`{ "data": { "id": "tsk_123", "previousStatus": "IN_PROGRESS", "status": "DONE", "boardColumnId": "col_done", "scoreTriggers": [ "compliance", "speed",`

	"activity"
	]
	},
	"meta": {
	"realtimePublished": true
	}
	}

Status codes	200, 403, 404, 409, 422, 500
Errores esperados	403; 404; 409 por transición inválida; 422 reglas de negocio

Validaciones clave

- Las transiciones se validan contra el workflow permitido del board.
- No se puede cerrar una tarea bloqueada si requiere checklist pendiente.
- `reason` es obligatorio si la política del board lo exige.

GET /api/v1/boards/:id · Get board detail

Propósito	Entregar columnas, tarjetas resumidas y configuración mínima del Kanban.
Auth requerida	Todos los roles autenticados con acceso al board.

Query / params	{
	"includeCards": "true|false",
	"assigneeId": "uuid opcional",
	"status": "filtro opcional"
	}

Request body	No aplica

Response ejemplo	{
	"data": {
	"id": "brd_ops",
	"name": "Operación diaria",
	"columns": [
	{
	"id": "col_todo",
	"name": "Por hacer",
	"order": 1,
	"cards": [
	{
	"id": "tsk_1",
	"title": "Abrir caja",
	"priority": "HIGH"
	}
	]
	},
	{
	"id": "col_done",
	"name": "Hecho",
	"order": 4,
	"cards": [
	{
	"id": "tsk_2",
	"title": "Checklist cierre",
	"priority": "MEDIUM"
	}
	]
	}
	]

	`},`
	`"meta": {`
	`"includeCards": true`
	`}`
	`}`

Status codes	200, 401, 403, 404, 500
Errores esperados	401; 403; 404

Validaciones clave

- El board debe pertenecer al workspace actual.
- Los filtros solo pueden afectar tarjetas visibles para el rol.

POST /api/v1/boards/:id/columns · Create board column

Propósito	Agregar una nueva columna configurable al tablero Kanban.
Auth requerida	Roles: admin, gerente.

Request body	`{`
	`"name": "Validación gerente",`
	`"colorToken": "violet",`
	`"wipLimit": 12,`
	`"order": 3`
	`}`

Response ejemplo	`{`
	`"data": {`
	`"id": "col_874",`
	`"name": "Validación gerente",`
	`"order": 3`
	`},`
	`"meta": {`
	`"boardId": "brd_ops"`
	`}`
	`}`

Status codes	201, 403, 404, 409, 422, 500
Errores esperados	403; 404; 409 por orden duplicado; 422

Validaciones clave

- `name` requerido, máximo 60 caracteres.
- `order` >= 1 y único dentro del board.
- `wipLimit` entre 1 y 999 cuando exista.

POST /api/v1/boards/:id/cards/reorder · Reorder board cards

Propósito	Persistir el nuevo orden de tarjetas y/o su cambio de columna luego de drag & drop.
Auth requerida	Roles: admin, gerente, supervisor.

Request body	`{`
	`"operations": [`
	`{`
	`"taskId": "tsk_1",`
	`"fromColumnId": "col_todo",`
	`"toColumnId": "col_doing",`
	`"toIndex": 0`

	},
	{
	"taskId": "tsk_5",
	"fromColumnId": "col_doing",
	"toColumnId": "col_doing",
	"toIndex": 1
	}
	]
	}

Response ejemplo	{
	"data": {
	"applied": 2,
	"realtimePublished": true
	},
	"meta": {
	"boardId": "brd_ops"
	}
	}

Status codes	200, 400, 403, 404, 409, 422, 500
Errores esperados	400; 403; 404; 409 por colisión de orden; 422

Validaciones clave

- `operations` debe traer entre 1 y 100 movimientos.
- Cada `taskId` debe pertenecer al board.
- Se rechazan huecos de orden o columnas inexistentes.

GET /api/v1/scores/summary · Score summary

Propósito	Entregar score actual, buckets, ranking y desglose por factor.
Auth requerida	Roles: admin, gerente, supervisor.

Query / params	{
	"range": "7d|28d",
	"userId": "uuid opcional",
	"teamId": "uuid opcional"
	}

Request body	No aplica

Response ejemplo	{
	"data": {
	"workspaceScore": 81.4,
	"distribution": {
	"high": 11,
	"medium": 5,
	"low": 2
	},
	"topPerformers": [
	{
	"userId": "usr_7",
	"name": "Andrea",
	"score": 94.1
	}
	],
	"factors": {
	"compliance": 84.0,
	"speed": 77.5,
	"consistency": 85.9,
	"activity": 80.2

	}
	},
	"meta": {
	"range": "28d",
	"snapshotAt": "2026-03-12T22:30:00Z"
	}
	}

Status codes	200, 401, 403, 404, 500
Errores esperados	401; 403; 404

Validaciones clave

- Si llega `userId`, debe pertenecer al workspace.
- `range` restringido a ventanas soportadas por el motor.

POST /api/v1/scores/recalculate · Recalculate score

Propósito	Forzar recálculo determinista para un usuario, equipo o workspace.
Auth requerida	Roles: admin, gerente.

Request body	{
	"scope": "workspace",
	"workspaceId": "wrk_1",
	"userId": null,
	"reason": "manual_admin_trigger"
	}

Response ejemplo	{
	"data": {
	"jobId": "job_score_992",
	"status": "queued"
	},
	"meta": {
	"acceptedAt": "2026-03-12T22:31:20Z"
	}
	}

Status codes	202, 400, 403, 409, 422, 500
Errores esperados	400; 403; 409 si ya hay job equivalente en curso; 422

Validaciones clave

- `scope` debe ser `user`, `team` o `workspace`.
- Se requiere identificador consistente con el scope.
- Se limita frecuencia para evitar tormenta de recálculos.

GET /api/v1/reports · List reports

Propósito	Listar reportes generados y su estado de disponibilidad.
Auth requerida	Roles: admin, gerente, supervisor.
Query / params	{ "type": "productivity|score|compliance", "status": "queued|processing|done|failed", "page": 1,

	"pageSize": 20 }
Request body	No aplica

Response ejemplo	{
	"data": [
	{
	"id": "rpt_11",
	"type": "score",
	"status": "done",
	"generatedAt": "2026-03-12T18:00:00Z",
	"downloadUrl": "https://.../rpt_11.pdf"
	}
	],
	"meta": {
	"page": 1,
	"total": 6
	}
	}

Status codes	200, 401, 403, 500
Errores esperados	401; 403; 500

Validaciones clave

- Filtros y paginación en rangos controlados.
- El downloadUrl solo se emite para artefactos accesibles por el rol.

POST /api/v1/reports/generate · Generate report

Propósito	Solicitar la generación de un reporte exportable en PDF.
Auth requerida	Roles: admin, gerente, supervisor.

Request body	{
	"type": "score",
	"range": {
	"from": "2026-03-01",
	"to": "2026-03-12"
	},
	"format": "pdf",
	"filters": {
	"teamId": "team_ops_1"
	}
	}

Response ejemplo	{
	"data": {
	"jobId": "job_rpt_222",
	"status": "queued"
	},
	"meta": {
	"etaHintSeconds": 20
	}
	}

Status codes	202, 400, 403, 409, 422, 500
Errores esperados	400; 403; 409 por job duplicado reciente; 422

Validaciones clave

- `type` soportado por el MVP.
- La fecha `from` no puede ser mayor a `to`.

- `format` en MVP es `pdf` únicamente.

GET /api/v1/checklists/instances · List checklist instances

Propósito	Listar checklists operativos activos o cerrados para una tarea, usuario o día.
Auth requerida	Todos los roles autenticados según ownership.
Query / params	`{ "taskId": "uuid opcional", "assigneeId": "uuid opcional", "status": "OPEN|DONE", "date": "2026-03-12" }`
Request body	No aplica
Response ejemplo	`{ "data": [ { "id": "chk_1", "title": "Checklist apertura", "status": "OPEN", "progress": { "done": 6, "total": 10 }, "taskId": "tsk_123" } ], "meta": { "total": 3 } }`
Status codes	200, 400, 401, 403, 500
Errores esperados	400; 401; 403

Validaciones clave

- Al menos un filtro de contexto cuando el rol es operador.
- La fecha debe cumplir formato ISO local de negocio.

PATCH /api/v1/checklists/instances/:id/items/:itemId · Toggle checklist item

Propósito	Marcar, desmarcar o comentar un item de checklist y disparar eventos de cumplimiento.
Auth requerida	Todos los roles con permiso sobre la instancia.
Request body	`{ "checked": true, "comment": "Caja validada y evidencia subida.", "evidenceUrl": "https://storage/.../photo.jpg" }`
Response ejemplo	`{ "data": { "instanceId": "chk_1", "itemId": "it_9", "checked": true, "progress": { "done": 7,`

	"total": 10
	}
	},
	"meta": {
	"scoreTriggers": [
	"compliance",
	"activity"
	]
	}
	}

Status codes	200, 403, 404, 409, 422, 500
Errores esperados	403; 404; 409 si la instancia está cerrada; 422

Validaciones clave

- `checked` es boolean requerido.
- El item debe pertenecer a la instancia.
- Si el template exige evidencia, `evidenceUrl` es obligatorio al cerrar.

GET /api/v1/alerts · List alerts

Propósito	Entregar bandeja de alertas in-app con filtros de lectura y prioridad.
Auth requerida	Todos los roles autenticados.

Query / params	{
	"status": "unread|read|all",
	"priority": "LOW|MEDIUM|HIGH|CRITICAL",
	"page": 1,
	"pageSize": 20
	}

Request body	No aplica

Response ejemplo	{
	"data": [
	{
	"id": "alt_1",
	"title": "Tarea vencida",
	"priority": "CRITICAL",
	"read": false,
	"createdAt": "2026-03-12T22:10:00Z",
	"relatedTaskId": "tsk_123"
	}
	],
	"meta": {
	"unreadCount": 4,
	"page": 1,
	"total": 12
	}
	}

Status codes	200, 401, 500
Errores esperados	401; 500

Validaciones clave

- `pageSize` entre 1 y 100.
- Solo se devuelven alertas visibles para el usuario/rol.

PATCH /api/v1/alerts/:id/read · Mark alert as read

Propósito	Marcar una alerta como leída desde topbar, panel lateral o centro de alertas.
Auth requerida	Todos los roles autenticados con ownership de la alerta.
Request body	`{ "read": true }`
Response ejemplo	`{ "data": { "id": "alt_1", "read": true, "readAt": "2026-03-12T22:34:00Z" }, "meta": { "updated": true } }`
Status codes	200, 401, 403, 404, 409, 500
Errores esperados	401; 403; 404; 409

Validaciones clave

- La alerta debe pertenecer al usuario o a un grupo visible por su rol.
- `read` solo acepta boolean.

8. Reglas transversales de ingeniería

- Todos los DTOs se implementan con Zod y se comparten entre cliente y servidor cuando sea posible.
- Los enums de negocio deben vivir en un paquete compartido (`@capataz/contracts`) para que UI, tests y backend no se contradigan.
- Cada mutación crítica escribe auditoría con `traceId`, `actorId`, `workspaceId`, `entityType`, `entityId` y `eventName`.
- Los jobs asíncronos como score y reportes responden `202 Accepted` con `jobId`; la UI no debe fingir que ya terminó el trabajo.
- La evolución futura hacia alertas multicanal, WhatsApp y fintech puede agregar módulos, pero no debe romper este contrato base sin versionado explícito.

9. Definición de terminado

- Existe catálogo cerrado de endpoints MVP.
- Cada endpoint crítico tiene request, response, validaciones, auth, errores y status codes.
- Server Actions y rutas HTTP apuntan al mismo contrato.
- QA puede derivar casos felices, bordes y errores sin adivinar semántica.
- El documento es suficiente para iniciar construcción backend y wiring frontend.